CS218: Homework 1

SJSU ID: 017415307

For this assignment, the application was developed in Golang. These are a few details about the code:

- The entrypoint for the code is the main package in the *main.go* file.
- The **'gin'** web framework of golang was employed for the backend.
- The logs are being written to a `temp/log/monitoring-api.log` file for future inspection.
- Authentication middleware is implemented to check the API_KEY passed in the request. This is present in the `middleware/auth.go` file in middleware folder
- Four different routes for getting CPU, Memory, Disk and Bandwidth have been implemented. The logic for this is written in the `handlers/handler.go` file.
- The server is listening on *port 8080*.
- A binary executable was created from this, which was then deployed to the EC2 and run from as a daemon with `systemctl` commands.

Steps for replicating this are as follows:

1. Download and unzip the `monitoring-api` folder.
2. In order to run the code locally for testing, download Go on your system.
3. Install dependencies with command: `go mod tidy`.
4. Create a binary file with the command: `go build -o monitoring-api`
5. Run the server with: `./monitoring-api`
6. Test this locally by running: `curl -H "API-KEY: cs218secret" http://localhost:8080/cpu`
7. If this is running well, we now start moving to EC2
8. The binary file created might not be appropriate for the EC2 instance so create one for linux architecture with command: `GOOS=linux GOARCH=amd64 go build -o monitoring-api`

9. Create a new EC2 instance of type `t2.micro` with `Amazon Linux OS`. Also create an RSA key-pair for access (or use a previous one). In this case the name of the file is `cs218demo.pem` and it is stored in the parent folder outside `monitoring-api`.

10. In order to allow for traffic from the internet on port 8080, add a new inbound rule to the EC2 security group.

    Click on EC2 instance ID >> Security Tab >> Click on Security Group >> Edit Inbound Rule >> Add Rule >> Type: Custom TCP, Port: 8080, Source: 0.0.0.0/0 >> Save Rule

11. Move the binary file from the local to EC2 with the command:

    ```
    scp -i ../../cs218demo.pem monitoring-api
    ec2-user@ec2-54-81-116-95.compute-1.amazonaws.com:/home/ec2-user/
    ```

    Replace the path to pem file with your path and the hostname(ec2-user@ec2-54-81-116-95.compute-1.amazonaws.com) with the Public IPv4 DNS of your EC2.

12. Connect to the EC2 with SSH: `ssh -i "../../cs218demo.pem" ec2-user@ec2-54-81-116-95.compute-1.amazonaws.com`

    The monitoring-api file should be visible by running the ls command inside EC2. Replace the hostname with yours.

13. Next create the folder for log file and give its access to ec2-user:

    ```
    mkdir -p /home/ec2-user/temp/log/

    chmod -R 755 /home/ec2-user/temp/log/

    chown -R ec2-user:ec2-user /home/ec2-user/temp/log/
    ```

14. Now you can try running the program manually in EC2 with command: `./monitoring-api`

15. To test the endpoint, call it from outside the EC2 (in a new terminal) with the command:
    `curl -H "API-KEY: cs218secret" https://ec2-54-81-116-95.compute-1.amazonaws.com:8080/cpu`

    Replace the hostname with yours.

16. Next to create a daemon from this, create a new service file with command:

    ```
    sudo nano /etc/systemd/system/monitoring-api.service
    ```

17. Add the following details to the file:

    ```
    [Unit]
    Description=Monitoring API Service
    After=network.target
    ```

```
[Service]
ExecStart=/home/ec2-user/monitoring-api
Restart=always
User=ec2-user
Environment=PATH=/usr/local/bin:/usr/bin:/bin
Environment=HOME=/home/ec2-user
WorkingDirectory=/home/ec2-user

[Install]
WantedBy=multi-user.target
```

18. Reload the daemons: `sudo systemctl daemon-reload`
19. Enable the new daemon created: `sudo systemctl enable monitoring-api`
20. Start the monitoring-api daemon: `sudo systemctl start monitoring-api`
21. Check the status with the command: `sudo systemctl status monitoring-api`

    The following should be its output

```
[ec2-user@ip-172-31-32-58 ~]$ sudo nano /etc/systemd/system/monitoring-api.service
[ec2-user@ip-172-31-32-58 ~]$ sudo systemctl daemon-reload
[ec2-user@ip-172-31-32-58 ~]$ sudo systemctl enable monitoring-api
[ec2-user@ip-172-31-32-58 ~]$ sudo systemctl start monitoring-api
[ec2-user@ip-172-31-32-58 ~]$ sudo systemctl status monitoring-api
● monitoring-api.service - Monitoring API Service
     Loaded: loaded (/etc/systemd/system/monitoring-api.service; enabled; preset: disabled)
     Active: active (running) since Thu 2024-10-10 05:24:01 UTC; 8s ago
   Main PID: 3696 (monitoring-api)
      Tasks: 4 (limit: 1112)
     Memory: 10.2M
        CPU: 9ms
     CGroup: /system.slice/monitoring-api.service
             └─3696 /usr/local/bin/monitoring-api

Oct 10 05:24:01 ip-172-31-32-58.ec2.internal systemd[1]: Started monitoring-api.service - Monitoring API Service.
[ec2-user@ip-172-31-32-58 ~]$ exit
logout
Connection to ec2-18-206-219-179.compute-1.amazonaws.com closed.
spartan@MLK-SCS-FVFGJ4HXQ6LR monitoring-api % curl -H "API-KEY: cs218secret" http://ec2-18-206-219-179.compute-1.amazonaws.com:8080/cpu
{"cpu_usage_percent":0.8886255924170474}%
spartan@MLK-SCS-FVFGJ4HXQ6LR monitoring-api %
```

22. Test the endpoint again to verify.
23. Its logs can be viewed by looking at the log file: `sudo nano /home/ec2-user/temp/log/monitoring-api.log`
24. Here is an example of some logs

```
2024/10/10 05:11:42 main.go:39: Starting server and listening on port:8080
[GIN] 2024/10/10 - 05:12:39 | 200 |      91.406µs |     98.210.6.62 | GET      "/cpu"
2024/10/10 05:24:02 main.go:39: Starting server and listening on port:8080
[GIN] 2024/10/10 - 05:24:20 | 200 |      98.389µs |     98.210.6.62 | GET      "/cpu"
[GIN] 2024/10/10 - 05:24:37 | 200 |     101.154µs |     98.210.6.62 | GET      "/cpu"
2024/10/10 06:06:06 auth.go:21: Unauthorized Access


^[[31m2024/10/10 06:06:06 [Recovery] 2024/10/10 - 06:06:06 panic recovered:
Unauthorized Access
/usr/local/go/src/log/log.go:439 (0x4e0d84)
/Users/spartan/Documents/Cloud/Assignment1/monitoring-api/middleware/auth.go:21 (0x76ed8e)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/context.go:185 (0x761d6e)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/recovery.go:102 (0x761d5b)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/context.go:185 (0x760ea4)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/logger.go:249 (0x760e8b)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/context.go:185 (0x76036a)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/gin.go:677 (0x760357)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/gin.go:670 (0x75fec4)
/Users/spartan/go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/gin.go:589 (0x75f991)
/usr/local/go/src/net/http/server.go:3210 (0x64fdcd)
/usr/local/go/src/net/http/server.go:2092 (0x6463cf)
/usr/local/go/src/runtime/asm_amd64.s:1700 (0x473100)
^[[0m
[GIN] 2024/10/10 - 06:06:06 | 401 |    1.108112ms |  87.120.166.244 | CONNECT  ""
```

25. Now the EC2 instance can be also stopped and restarted. The program will start running on restart.

Challenges Faced:

- I was learning Go for the first time but it has a good learning curve.
- Setting up the logging was a bit complex as it involved writing and appending to a file.
- Creating a daemon took some time as it was difficult to understand what went wrong when the service suddenly stopped running. After I found the command for viewing the logs this became easier: `sudo journalctl -u monitoring-api.service -b`
- I was hoping to also set up https for more secure communication but was facing too many errors. That remains an open point.

References:

- Learn GO Fast: Full Tutorial
- Tutorial: Developing a RESTful API with Go and Gin - The Go Programming Language
- Creating Rest API using Gin Framework (Episode #1)